CLASE 2 · 3 HS

Variables y Números

Que el programa **recuerde** y **calcule** números. Al salir, el entrenador cuenta de 0 a 15 y muestra cada número en decimal, en hexa y en binario con 4 LEDs.

HOY

Siete bloques, un contador en binario

1 · 10 min

Repaso y una pregunta

2 · 30 min

Variables

3 · 30 min

Operaciones

4 · 10 min

const para los pines

5 · 35 min

Decimal a binario

6 · 25 min

Contar y desbordar

7 · 40 min

Ejercicios

TP

Mi entrenador cuenta en binario

¿Cómo hacemos que el ESP32 cuente 0, 1, 2, 3…?

```
Serial.printf("0\n");
Serial.printf("1\n");
Serial.printf("2\n");
Serial.printf("3\n");
// … ¿y hasta mil?
```

En la Clase 1 el ESP32 repetía siempre lo mismo. No podía **acordarse** de nada ni **hacer cuentas**.

Necesitamos un lugar donde **guardar** el número y poder **cambiarlo**. Eso es una **variable**.

Una variable es una cajita en la memoria



Parte	Qué es	Ejemplo
Tipo	Qué clase de cosa entra y cuánto lugar ocupa	`int`
Nombre	Cómo se la llama en el programa	`edad`
Valor	Lo que tiene guardado ahora	19

Letras, números y **_**. No empieza con número ni tiene espacios.

Sin tildes ni ñ: **anio** sí, **año** no.

Edad y **edad** son distintas. **contador** dice más que **x**.

Declarar, asignar y el = de C

```
int edad = 19;       // DECLARAR y ASIGNAR en una línea
int anio;            // DECLARAR: crear la cajita (tipo + nombre)
anio = 2026;         // ASIGNAR: guardar 2026 en la cajita

Serial.printf("Tengo %d anios\n", edad);
edad = edad + 1;     // leer edad (19), sumarle 1, guardar 20
Serial.printf("El anio que viene voy a tener %d\n", edad);
```



El **=** de C **no es** el igual de matemática: significa **guardar lo de la derecha en la cajita de la izquierda**. En matemática x = x + 1 no tiene solución; en C es lo más común del mundo.

Los tres tipos que vamos a usar

Tipo	Qué guarda	Rango	Ejemplo
`int`	Entero con signo (32 bits en el ESP32)	−2.147.483.648 a 2.147.483.647	`int edad = 19;`
`uint8_t`	Entero **sin signo** de **8 bits**	**0 a 255**	`uint8_t nota = 8;`
`float`	Número con coma (con **punto**: 27.5)	~7 cifras de precisión	`float temp = 27.5;`

uint8_t = *unsigned int* de 8 bits. Con **n bits** hay **2^n** valores: $2^8 = 256$, del 0 al 255. Un uint8_t es **un byte**, y la placa tiene **8 LEDs**: un LED por bit.

Imprimir variables con printf

```
Serial.printf("Tengo %d anios\n", edad);   →   Tengo 19 anios
```

el %d se reemplaza por el valor de edad

Formato	Imprime	Ejemplo	Sale
%d	Entero en **decimal** (con signo)	printf("%d", 42)	42
%u	Entero en decimal **sin signo**	printf("%u", nota)	8
%X	Entero en **hexadecimal** (mayúsculas)	printf("%X", 255)	FF
%f	Número con coma	printf("%f", 27.5)	27.500000
%.2f	Con coma, **2 decimales**	printf("%.2f", 27.5)	27.50
%%	El carácter %	printf("100%%")	100%
%3u	Al menos 3 lugares, rellena con espacios	printf("%3u", 7)	7
%02X	Al menos 2 dígitos, rellena con **ceros**	printf("%02X", 7)	07

Los operadores, con a = 17 y b = 5

Operador	Qué hace	Ejemplo	Resultado
+	Suma	a + b	22
-	Resta	a - b	12
*	Multiplicación	a * b	85
/	División	a / b	**3** (¡ojo!)
%	**Resto** de la división (módulo)	a % b	2
++	Suma 1 a la variable	a++;	a pasa a valer 18

Mensaje del bloque: **entre enteros, la división corta. Lo que sobra lo da el %.**

División entera: corta, no redondea

```
17 | 5
 2   3
```

```
17 / 5 = 3   cociente
17 % 5 = 2   resto
3 × 5 + 2 = 17
```

```
17 / 5  → 3   no 3,4
 7 / 2  → 3   no 3,5
 1 / 2  → 0
```

Para decimales, **float**: 17.0 / 5.0 con %.2f da **3.40**.

n % 2 Vale **0 si n es par** y **1 si n es impar**.

n % 16 Siempre da **entre 0 y 15**: un contador que "da la vuelta".

Primero en papel, después en la placa

```
int a = 17;
int b = 5;
Serial.printf("a / b = %d\n", a / b);
Serial.printf("a %% b = %d\n", a % b);
float x = 17.0;
float y = 5.0;
Serial.printf("x / y = %.2f\n", x / y);
int n = 255;
Serial.printf("%d en hexa es %X\n", n, n);
n++;
Serial.printf("Despues de n++, n vale %d (hexa %X)\n", n, n);
```

Predecir

Anotar qué imprime cada línea.

Después correr y comparar.

¿Qué LED era el 17? Mejor ponerle nombre

```
const int L0 = 4;
const int L1 = 16;
const int L2 = 17;
const int L3 = 18;

void setup() {
  pinMode(L2, OUTPUT);
  digitalWrite(L2, HIGH);  // "encender L2"
}
```

const = constante: una variable que **no se puede cambiar**.

```
L0 = 5;
```

```
assignment of read-only variable 'L0'
```

Van **arriba de todo**, afuera de setup() y loop(), para usarlas en los dos.



El 13 en binario: en papel y en C

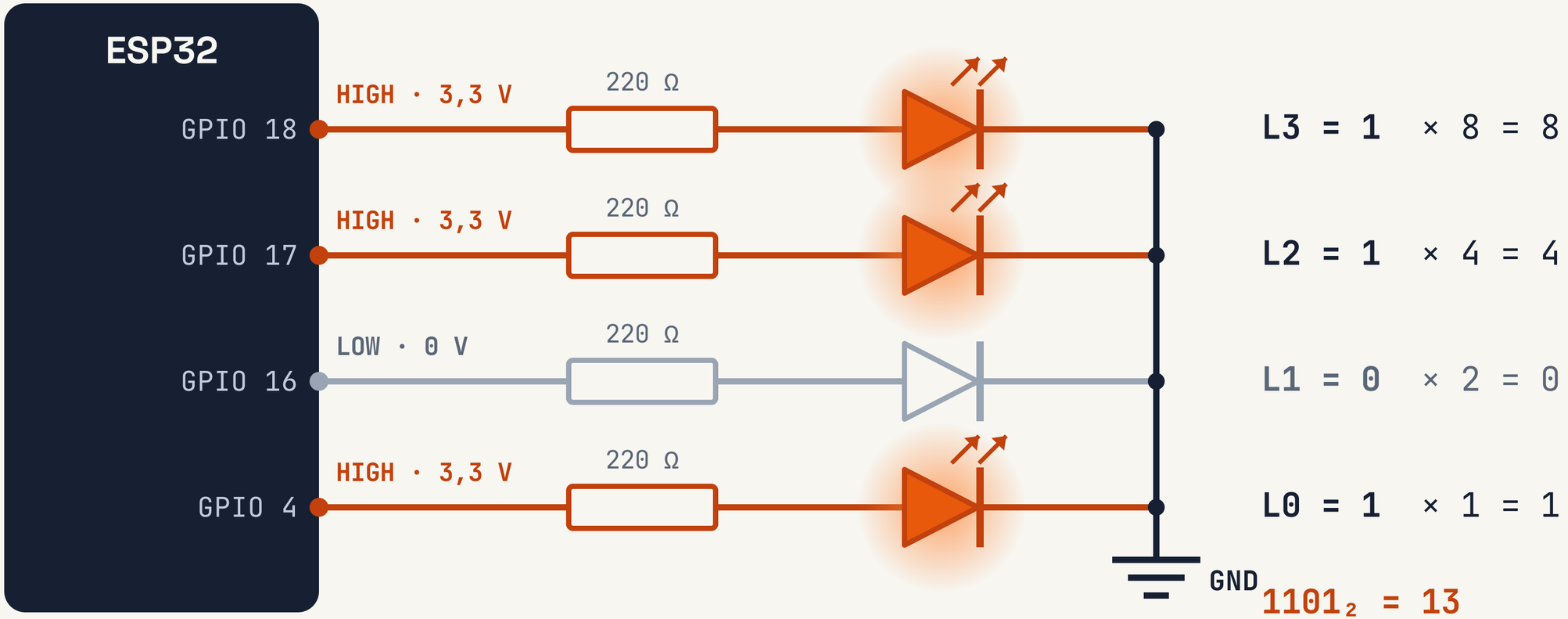
```
13 / 2 = 6   resto 1   bit 0
 6 / 2 = 3   resto 0   bit 1
 3 / 2 = 1   resto 1   bit 2
 1 / 2 = 0   resto 1   bit 3
```

13 = 1101, leído de abajo hacia arriba.

Bit	Peso	En C	Para n = 13
bit 0	1	`n % 2`	`13 % 2 =` **1**
bit 1	2	`(n / 2) % 2`	`6 % 2 =` **0**
bit 2	4	`(n / 4) % 2`	`3 % 2 =` **1**
bit 3	8	`(n / 8) % 2`	`1 % 2 =` **1**

Dividir por el **peso** del bit corre el número hasta que ese bit queda en las unidades, y **% 2** lo extrae. Dividir dos veces por 2 es dividir por 4: (13 / 2) / 2 = 13 / 4 = 3.

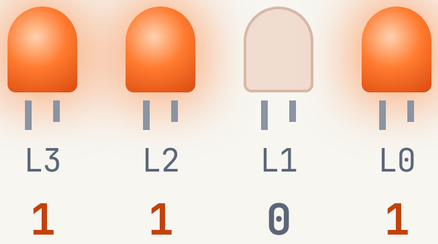
Así se conecta cada LED (con n = 13)



Cada cuenta da 0 o 1: justo LOW o HIGH

```
int n = 13;   // probar de 0 a 15

digitalWrite(L0, n % 2);
digitalWrite(L1, (n / 2) % 2);
digitalWrite(L2, (n / 4) % 2);
digitalWrite(L3, (n / 8) % 2);
```



= 1101_2 = 13

Decimal	Binario (L3 L2 L1 L0)	Hexa
0	0000	0
5	0101	5
9	1001	9
10	1010	A
12	1100	C
15	1111	F

4 bits son un dígito hexa: $1111_2 = 15 = F_{16}$. Los 4 LEDs L0–L3 forman un *nibble*.

¿Dónde declaro el contador?

Adentro de loop(): imprime 0, 0, 0… para siempre

```
void loop() {
  int contador = 0;  // en CADA vuelta
  Serial.printf("%d\n", contador);
  contador++;
  delay(1000);
}
```

Afuera, arriba de todo: imprime 0, 1, 2, 3…

```
int contador = 0;  // una sola vez

void loop() {
  Serial.printf("%d\n", contador);
  contador++;
  delay(1000);
}
```

```
contador = (contador + 1) % 16;
```

14 → 15 → 0 → 1

Un uint8_t que vale 255, más 1, da 0

```
Decimal: 254   Hexa: FE
Decimal: 255   Hexa: FF
Decimal:   0   Hexa: 00
Decimal:   1   Hexa: 01
```

```
  11111111
+        1
 ----------
 100000000
```

Da **9 bits**, pero la cajita tiene 8: el **acarreo** de la izquierda **se pierde**. Queda 00000000 = 0.

La aritmética de **n bits** es aritmética **módulo 2^n**: un uint8_t cuenta módulo 256, como un cuentakilómetros. El bit perdido es el **carry out** del sumador de 8 bits. Y no da error ni aviso.

7 · EJERCICIOS · 40 MIN

Ahora ustedes

EJERCICIO 1

Calculadora

Con **int a** e **int b**: suma, resta, producto, cociente y resto, como **23 / 4 = 5 (resto 3)**. Agregar la división con float. Probar con b = 0: ¿qué pasa?

EJERCICIO 2

Horas, minutos y segundos

3725 segundos en formato **01:02:05**, solo con / y %. Una hora tiene 3600 s; %02d da dos dígitos. Desafío: que avance como un reloj.

EJERCICIO 3

Verificación a mano

Sin la computadora, pasar 11 a binario. Después mostrarlo en L0–L3 y comparar. Repetir con 6 y 15. Imprimir 8*b3 + 4*b2 + 2*b1 + 1*b0.

Los que terminan rápido arrancan con el TP y su extensión.

TP CLASE 2

Mi entrenador cuenta en binario

1. Partir del TP de la Clase 1, con el cartel de presentación.
2. Comentario de encabezado actualizado: Clase 2 y qué hace ahora.
3. Constantes para los pines: const int L0 = 4; etc.
4. Un contador de 0 a 15 que avanza 1 por segundo y vuelve a 0.
5. El contador en binario en L0–L3 (L0 = bit de menos peso).
6. Cada valor impreso en decimal y en hexa.

```
Decimal:  9  Hexa: 9  → LEDs 1001
Decimal: 10  Hexa: A  → LEDs 1010
Decimal: 11  Hexa: B  → LEDs 1011
```

Extensión (opcional, suma puntos): un contador uint8_t de 0 a 255 en los 8 LEDs, impreso en decimal y en hexa con %02X. Sin % 256: que desborde solo y se vea el paso de FF a 00. Para no esperar 4 minutos, arrancar en 250.

Entrega: carpeta **SL2026-Apellido-Clase2** en ZIP, sin la carpeta .pio.



CHECKLIST DE SALIDA

Al terminar la clase, sé…

- explicar qué es una variable (tipo, nombre, valor) y la diferencia entre declarar y asignar
- elegir entre int, uint8_t y float, y decir el rango de un uint8_t
- imprimir variables con %d, %u, %X y %f
- predecir el resultado de 17 / 5 y de 17 % 5
- usar const int para darle nombre a un pin
- pasar un número de 0 a 15 a binario con divisiones sucesivas, en papel y en C
- explicar por qué un uint8_t que vale 255 pasa a 0 al sumarle 1

La semana que viene: hoy el programa hace cuentas, pero siempre hace lo mismo. En la Clase 3 va a **decidir**: leer los pulsadores y convertir el ESP32 en compuertas AND, OR y XOR.